Experiment 6

Aim : Apply K-Means and Hierarchical Clustering on sample datasets

Dataset : clustering_experiment_data.csv

Theory :

Code and Outputs :

Step 1 : Implement K-Means Clustering

```
from sklearn.cluster import KMeans

# Initialize KMeans with 4 clusters and random_state 42
kmeans = KMeans(n_clusters=4, random_state=42, n_init='auto')

# Fit the model and predict cluster assignments
df['KMeans_Labels'] = kmeans.fit_predict(df[['Feature 1', 'Feature 2']])

# Display the first few rows to verify labels
print("Dataset with K-Means labels:")
print(df.head())
```

Output :

```
Dataset with K-Means labels:
   Feature 1  Feature 2  KMeans_Labels
0  -9.297689   6.473679              3
1  -9.698741   6.938967              3
2  -1.686653   7.793442              0
3  -7.097308  -5.781333              1
4 -10.876452   6.315437              3
```

Step 2 : Implement Hierarchical Clustering

```
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt
```

SubStep 1. Create a linkage matrix using the 'ward' method

```
Z = linkage(df[['Feature 1', 'Feature 2']], method='ward')
```

SubStep 2. Plot the dendrogram

```
plt.figure(figsize=(10, 7))
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Sample Index')
```

```
plt.ylabel('Distance (Ward)')
dendrogram(Z, leaf_rotation=90., leaf_font_size=8.)
plt.show()
```

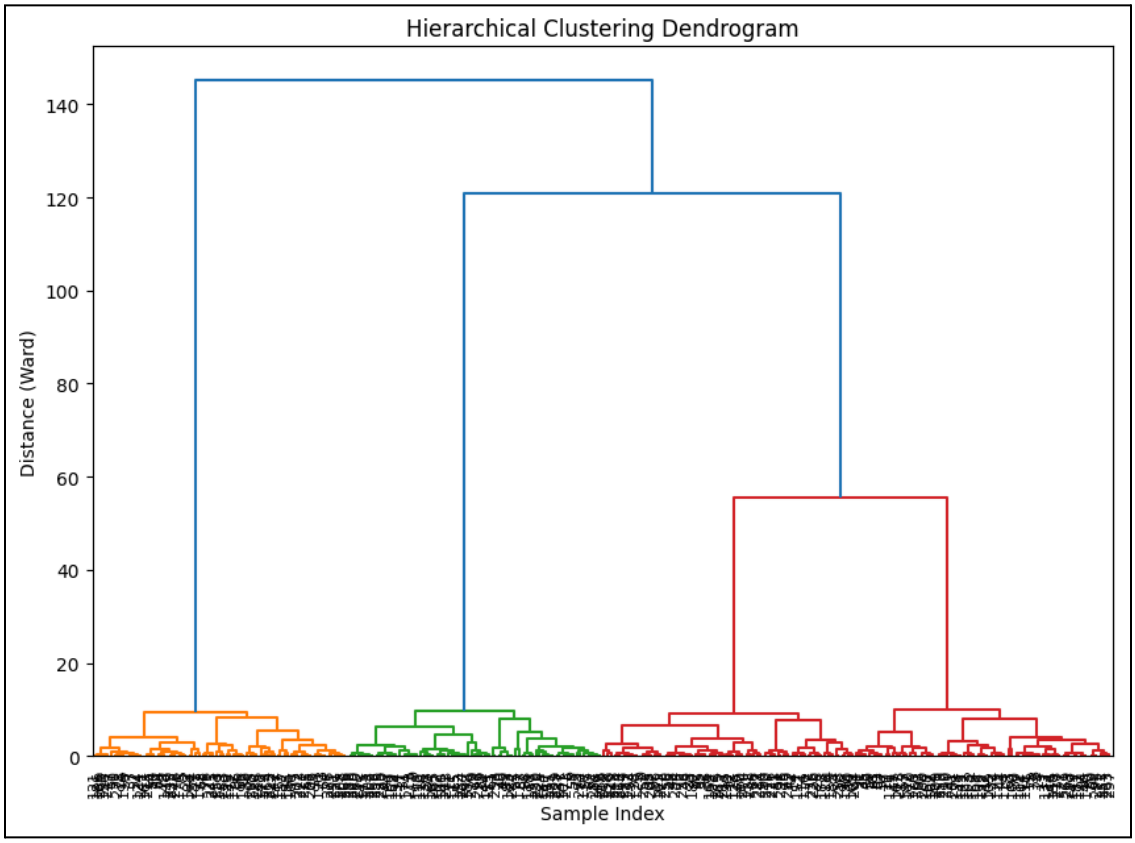
SubStep 3. Instantiate and fit AgglomerativeClustering

```
agg_clustering = AgglomerativeClustering(n_clusters=4)
df['Hierarchical_Labels'] = agg_clustering.fit_predict(df[['Feature 1', 'Feature 2']])
```

SubStep 4. Display the first few rows

```
print("Dataset with Hierarchical Clustering labels:")
print(df.head())
```

Output :




```
Dataset with Hierarchical Clustering labels:
   Feature 1  Feature 2  KMeans_Labels  Hierarchical_Labels
0  -9.297689   6.473679              3                    3
1  -9.698741   6.938967              3                    3
2  -1.686653   7.793442              0                    0
3  -7.097308  -5.781333              1                    1
4 -10.876452   6.315437              3                    3
```

Step 3 : Visualize and Compare the results

```
import matplotlib.pyplot as plt

# Create a figure with two side-by-side subplots
fig, axes = plt.subplots(1, 2, figsize=(15, 6))
```

SubStep 1. Plot K-Means Results

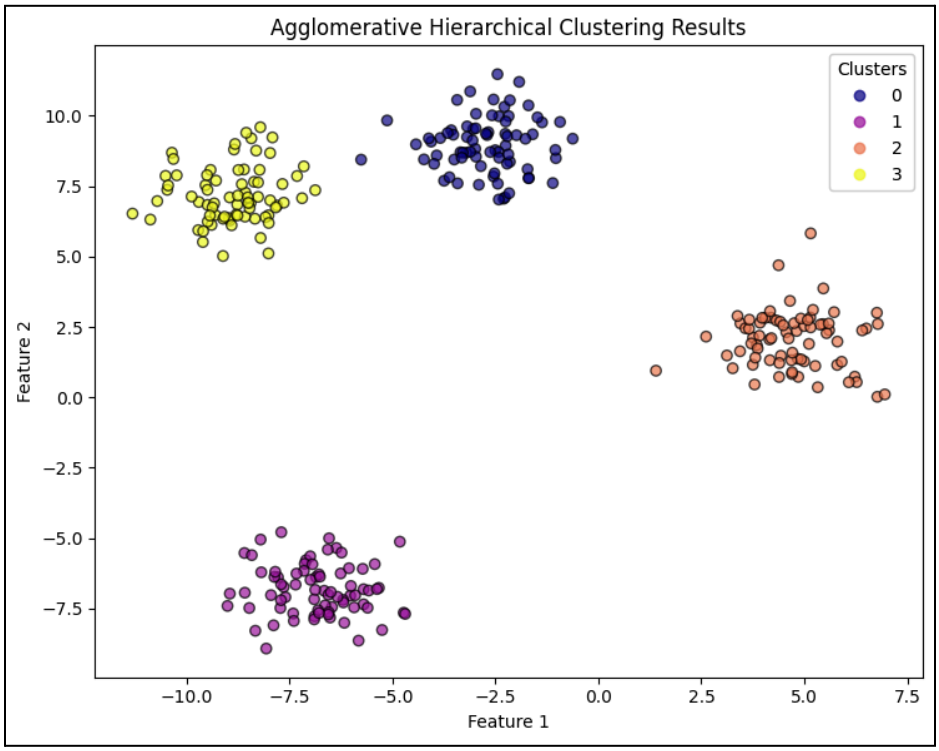
```
scatter1 = axes[0].scatter(df['Feature 1'], df['Feature 2'], c=df['KMeans_Labels'], cmap='viridis',
edgecolors='k', alpha=0.7)
axes[0].set_title('K-Means Clustering Results')
axes[0].set_xlabel('Feature 1')
axes[0].set_ylabel('Feature 2')
legend1 = axes[0].legend(*scatter1.legend_elements(), title="Clusters")
axes[0].add_artist(legend1)
```

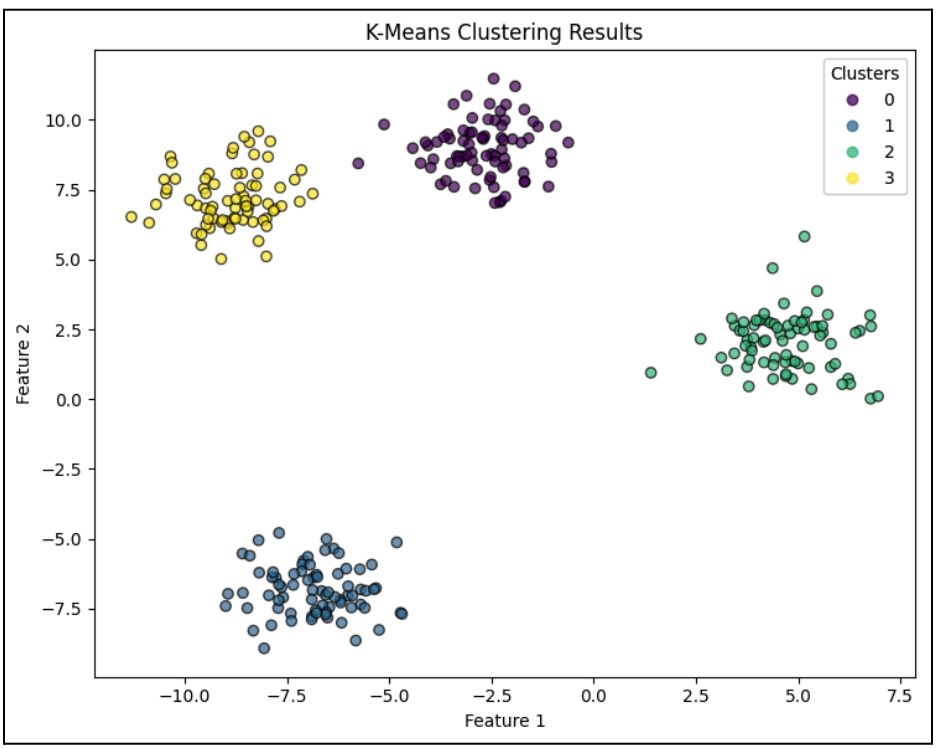
Sub Step 2. Plot Hierarchical Clustering Results

```
scatter2 = axes[1].scatter(df['Feature 1'], df['Feature 2'], c=df['Hierarchical_Labels'],
cmap='plasma', edgecolors='k', alpha=0.7)
axes[1].set_title('Agglomerative Hierarchical Clustering Results')
axes[1].set_xlabel('Feature 1')
axes[1].set_ylabel('Feature 2')
legend2 = axes[1].legend(*scatter2.legend_elements(), title="Clusters")
axes[1].add_artist(legend2)

# Adjust layout and display
plt.tight_layout()
plt.show()
```

Output :





Conclusion : Both methods successfully identified the four distinct clusters generated in the synthetic dataset. While K-Means relies on centroid-based partitioning, the Agglomerative approach uses a bottom-up hierarchical strategy (specifically the Ward linkage method). On this well-separated dataset, both algorithms produced nearly identical cluster boundaries.